

CS560 - UI/UX Design & Implementation

4-2 Activity: Implementing Reusable Components and Logic

Malgorzata Debska

Southern New Hampshire University

Nathan Palmer

May 13, 2026

Reusable Components and Logic Reflection

This assignment helped me better understand the importance of reusable components and reusable logic in modern front-end development. Instead of creating separate layouts and code for each page, I implemented a reusable ProductCard component that could be shared across multiple parts of the application. This approach helped create a cleaner, more organized, and more maintainable project structure. It also ensured that the product information appeared in a consistent format throughout the application, which improves the overall user experience.

The reusable ProductCard component was designed to display product details such as the product name, category, description, price, and additional specifications. On the All Products page, the component included an Add to Cart button that allowed users to place products into the shopping cart. On the Shopping Cart page, the same reusable component was used again, but with different functionality. Instead of only showing an Add to Cart button, the component displayed controls that allowed users to increase the quantity, decrease the quantity, and remove products from the cart. Reusing the same component on multiple pages helped reduce duplicate code and simplify the development process.

Using reusable components supports flexibility because the component can easily be modified or expanded without rewriting large sections of the application. For example, if the company later wanted to add product ratings, product images, or inventory tracking, those features could be added directly to the reusable component and would automatically appear everywhere the component is used. This makes future updates easier and helps the application scale more efficiently as additional features are added.

This assignment also demonstrated the importance of maintainability. Keeping the code centralized in reusable components makes the application easier to read, debug, and manage. Instead of updating multiple files with repeated code, changes can be made in one location. This reduces the chance of inconsistencies or errors across different pages. The reusable logic for the shopping cart also improved maintainability because the cart totals, quantities, and product updates were handled dynamically through shared functionality instead of separate hardcoded solutions.

Another important benefit of this approach is scalability. Modern web applications often contain hundreds of components and features. Organizing applications into reusable components allows developers to build larger systems more efficiently. This assignment showed how component-based design can support long-term application growth while maintaining a consistent visual design and user experience.

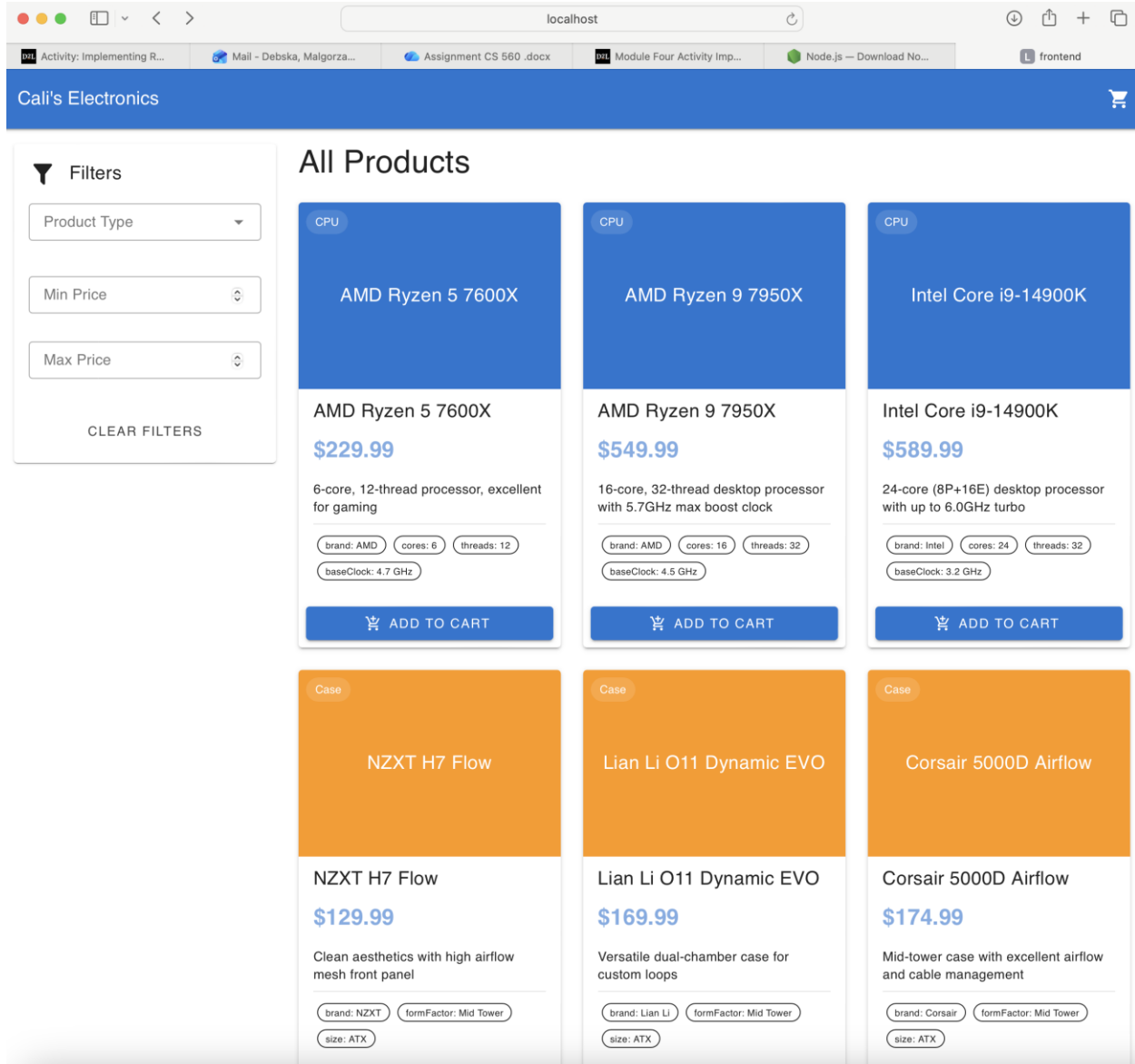
Throughout the implementation process, I tested the application multiple times to ensure that all interactions worked correctly. I verified that the Add to Cart button added the correct products to the shopping cart, the quantity buttons updated totals properly, and the remove button successfully removed items from the cart while updating the totals dynamically. Testing each interaction helped confirm that the reusable component and cart logic functioned as intended.

To conclude, this assignment strengthened my understanding of component-based development and reusable logic in front-end frameworks. Learning how to create reusable components prepares me for future software and application development because modern frameworks such as Vue, React, and Angular rely heavily on reusable structures. This experience will help me

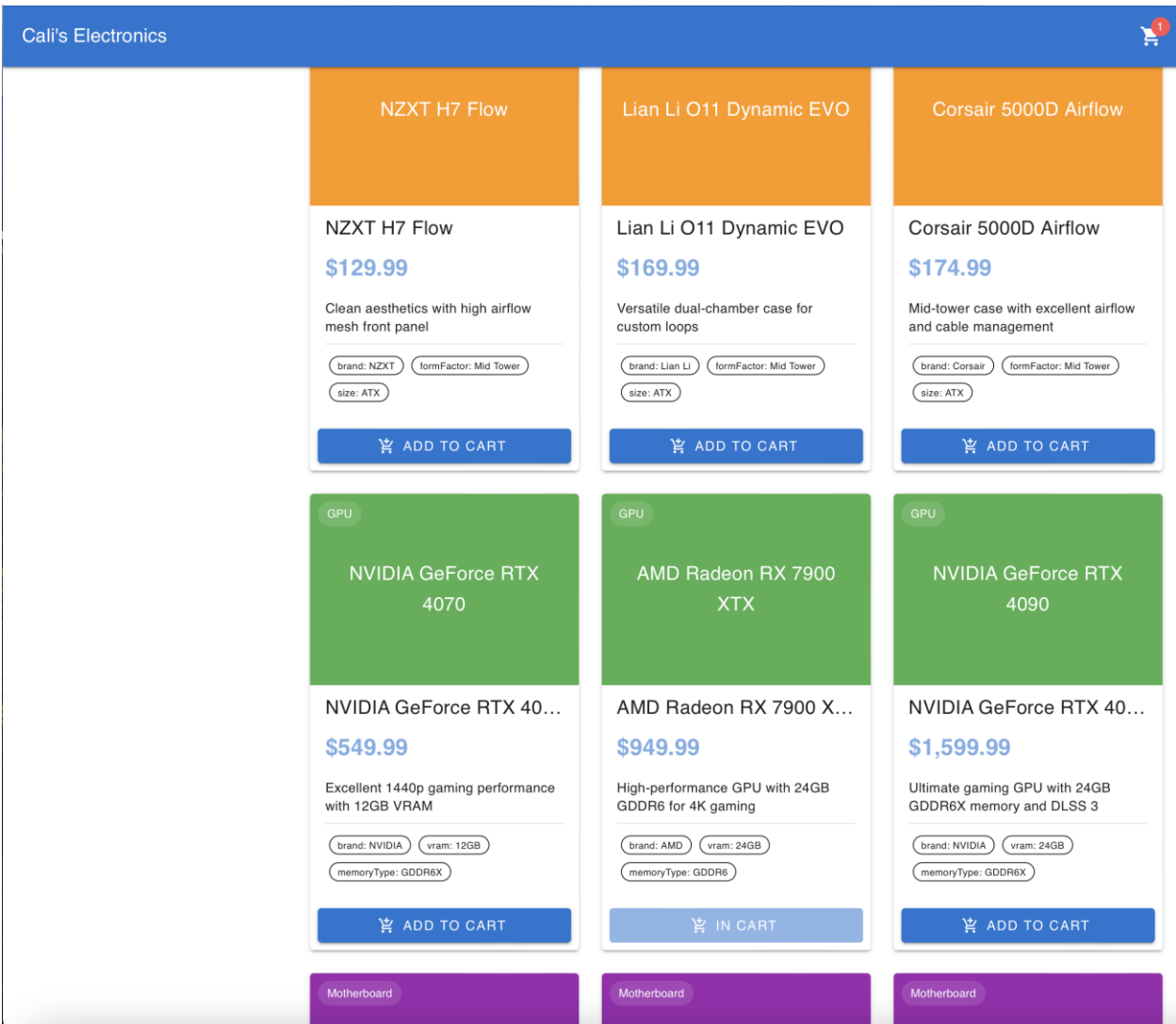
develop larger and more scalable applications in future academic and professional projects while maintaining clean, organized, and efficient code.

Screenshots of the working website:

All products



Add to cart :



View in the cart:

Shopping Cart

GPU

AMD Radeon RX 7900 XTX

AMD Radeon RX 7900 XTX

\$949.99

High-performance GPU with 24GB GDDR6 for 4K gaming

brand: AMD

ram: 24GB

memoryType: GDDR6

-

1

+

Order Summary

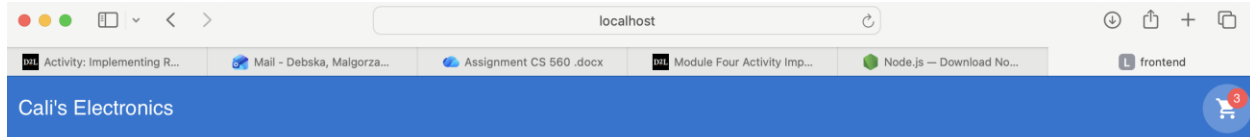
Total:

\$949.99

[CLEAR CART](#)

[CHECKOUT](#)

Increasing the items in the cart:



Shopping Cart

GPU

AMD Radeon RX 7900 XTX

AMD Radeon RX 7900 XTX

\$949.99

High-performance GPU with 24GB GDDR6 for 4K gaming

brand: AMD

vram: 24GB

memoryType: GDDR6

-

3

+

Order Summary

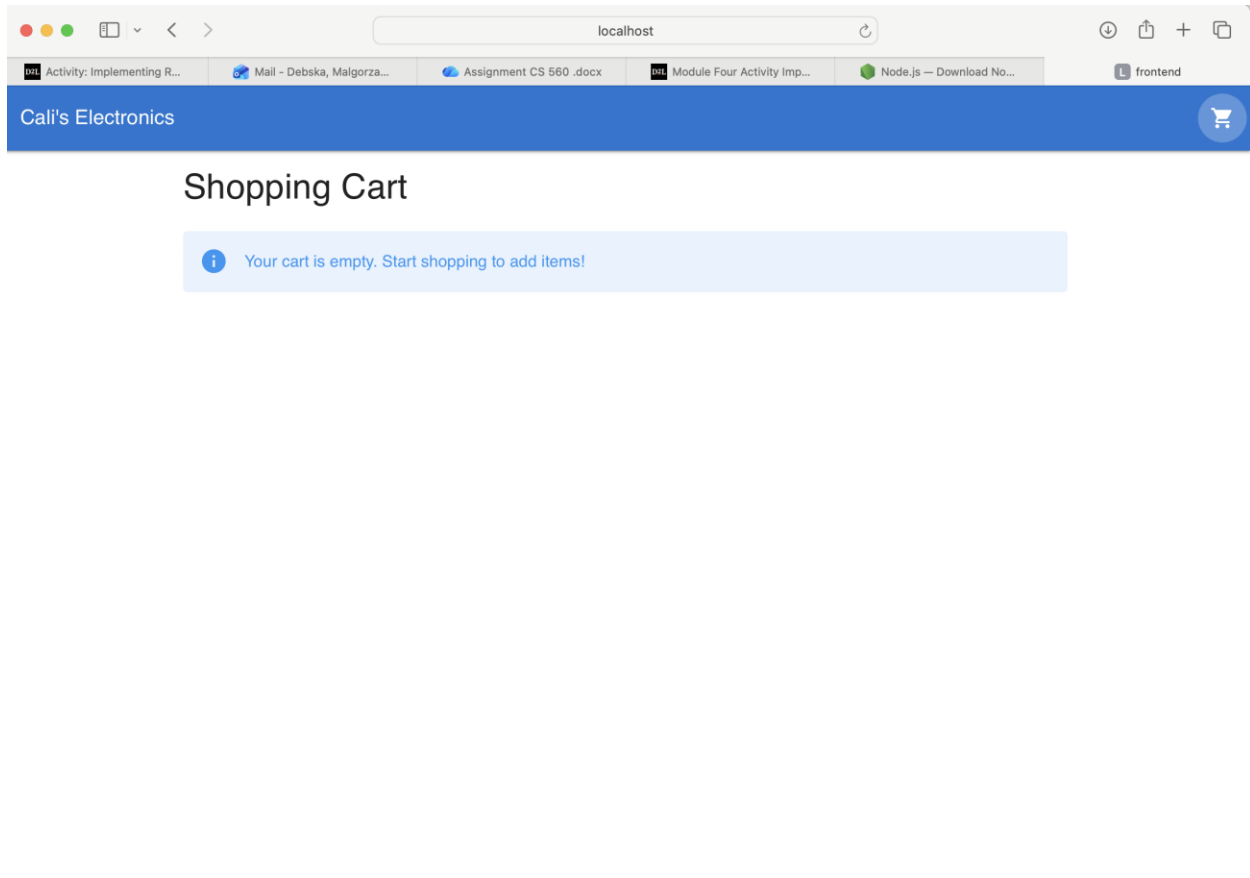
Total:

\$2,849.97

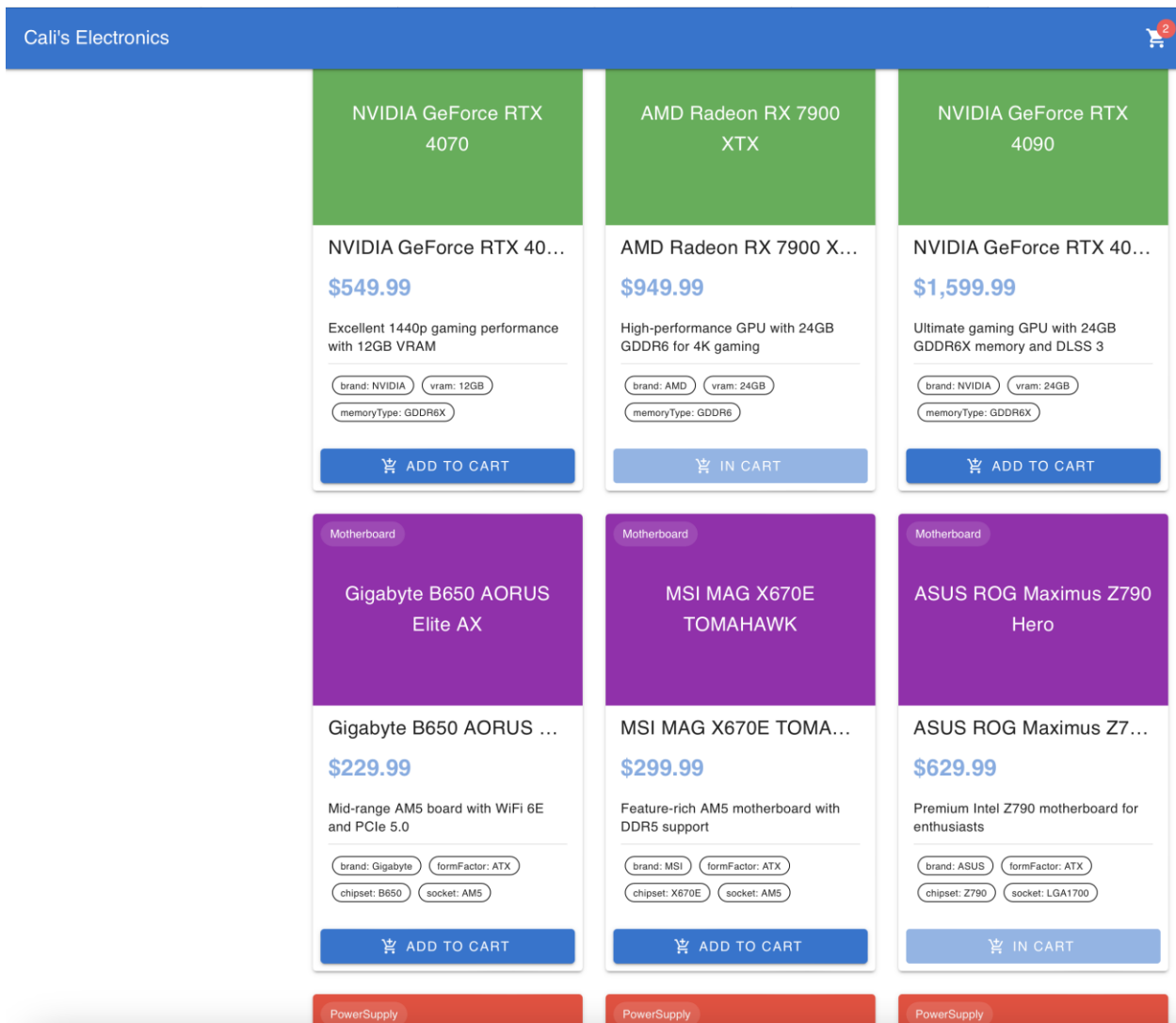
[CLEAR CART](#)

[CHECKOUT](#)

Clearing the cart:



Example 2: Adding to the cart 2 different items



Increasing the items in the cart:

Shopping Cart

GPU	Motherboard
AMD Radeon RX 7900 XTX	ASUS ROG Maximus Z790 Hero
\$949.99	\$629.99
High-performance GPU with 24GB GDDR6 for 4K gaming	Premium Intel Z790 motherboard for enthusiasts
brand: AMD vram: 24GB memoryType: GDDR6	brand: ASUS formFactor: ATX chipset: Z790 socket: LGA1700
- 3 + 	- 3 + 

Order Summary

Total: **\$4,739.94**

[CLEAR CART](#)[CHECKOUT](#)

Decreasing the items in the cart:

Shopping Cart

GPU

AMD Radeon RX 7900 XTX

AMD Radeon RX 7900 XTX

\$949.99

High-performance GPU with 24GB GDDR6 for 4K gaming

brand: AMD vram: 24GB memoryType: GDDR6

- 2 +

Motherboard

ASUS ROG Maximus Z790 Hero

ASUS ROG Maximus Z790 Hero

\$629.99

Premium Intel Z790 motherboard for enthusiasts

brand: ASUS formFactor: ATX chipset: Z790 socket: LGA1700

- 1 +

Order Summary

Total:

\$2,529.97

CLEAR CART

CHECKOUT

Clearing the cart:

Shopping Cart

 Your cart is empty. Start shopping to add items!

Code Screenshots:

ProductCard.vue

{ launch.json

▼ ProductCard.vue ×

frontend > src > components > ▼ ProductCard.vue > ...

```
1  <script setup lang="ts">
2  // =====
3  // ProductCard.vue - Reusable Product Card Component
4  // =====
5  // This reusable component centralizes the product card layout and behavior for
6  // both the All Products page and the Shopping Cart page.
7  // =====
8
9  import { computed } from 'vue';
10 import type { Product } from '../types';
11 import { useCart } from '../composables/useCart';
12
13 interface Props {
14   product: Product;
15   showAddToCart?: boolean;
16   showQuantityControls?: boolean;
17   quantity?: number;
18 }
19
20 const props = withDefaults(defineProps<Props>(), {
21   showAddToCart: true,
22   showQuantityControls: false,
23   quantity: 0,
24 });
25
26 const emit = defineEmits<{
27   (e: 'remove'): void;
28   (e: 'update:quantity', value: number): void;
29 }>();
30
31 const { addToCart, isInCart } = useCart();
32
33 const typeColor = computed(() => {
34   const colors: Record<string, string> = {
35     CPU: '#1976D2',
36     GPU: '#4CAF50',
37     Motherboard: '#9C27B0',
38     Case: '#FF9800',
39     PowerSupply: '#F44336',
40     RAM: '#009688',
41   };
42   return colors[props.product.type] || '#757575';
43 });
44
45 const formattedPrice = computed(() => {
46   return new Intl.NumberFormat('en-US', {
47     style: 'currency',
48     currency: 'USD',
49   }).format(props.product.price);
50 });
51
52 const specsList = computed(() => {
53   return Object.entries(props.product.specs)
54     .filter(([, value]) => value !== undefined)
55     .slice(0, 4);
56 });
57
58 const productIsInCart = computed(() => isInCart(props.product.id));
59
60 const handleAddToCart = () => {
61   addToCart(props.product);
62 };
```

```
{} launch.json  ProductCard.vue X
frontend > src > components > ProductCard.vue > ...
1  <script setup lang="ts">
60  const handleAddToCart = () => {
62  };
63
64  const incrementQuantity = () => {
65    emit('update:quantity', props.quantity + 1);
66  };
67
68  const decrementQuantity = () => {
69    if (props.quantity > 1) {
70      emit('update:quantity', props.quantity - 1);
71    } else {
72      emit('remove');
73    }
74  };
75  </script>
76
77  <template>
78    <v-card class="h-100 d-flex flex-column" elevation="2">
79      <div
80        class="d-flex align-center justify-center position-relative"
81        :style="{ backgroundColor: typeColor, height: '200px' }">
82        >
83        <span class="text-h6 text-white text-center px-4">{{ product.name }}</span>
84        <v-chip
85          color="white"
86          class="ma-2 position-absolute"
87          style="top: 0; left: 0"
88          size="small"
89        >
90          {{ product.type }}
91        </v-chip>
92      </div>
93
94      <v-card-title class="text-h6">
95        {{ product.name }}
96      </v-card-title>
97
98      <v-card-subtitle class="text-h5 font-weight-bold text-primary">
99        {{ formattedPrice }}
100      </v-card-subtitle>
101
102      <v-card-text class="flex-grow-1">
103        <p class="text-body-2 mb-2">{{ product.description }}</p>
104
105        <v-divider class="my-2" />
106
107        <div class="d-flex flex-wrap">
108          <v-chip
109            v-for="[key, value] in specsList"
110            :key="key"
111            size="x-small"
112            variant="outlined"
113            class="ma-1"
114          >
115            {{ key }}: {{ value }}
116          </v-chip>
117        </div>
118      </v-card-text>
119
120      <v-card-actions v-if="showAddToCart || showQuantityControls">
121        <v-btn
122          v-if="showAddToCart"
123          color="primary"
```

launch.json

ProductCard.vue X

frontend > src > components > ProductCard.vue > ...

```
77 <template>
78   <v-card class="h-100 d-flex flex-column" elevation="2">
102     <v-card-text class="flex-grow-1">
107       <div class="d-flex flex-wrap">
108         <v-chip
112           variant="outlined"
113           class="ma-1"
114         >
115           {{ key }}: {{ value }}
116         </v-chip>
117       </div>
118     </v-card-text>
119
120     <v-card-actions v-if="showAddToCart || showQuantityControls">
121       <v-btn
122         v-if="showAddToCart"
123         color="primary"
124         variant="elevated"
125         block
126         :disabled="productIsInCart"
127         @click="handleAddToCart"
128       >
129         <v-icon start>mdi-cart-plus</v-icon>
130         {{ productIsInCart ? 'In Cart' : 'Add to Cart' }}
131       </v-btn>
132
133       <div
134         v-if="showQuantityControls"
135         class="d-flex align-center w-100 justify-center"
136       >
137         <v-btn
138           icon="mdi-minus"
139           size="small"
140           variant="outlined"
141           @click="decrementQuantity"
142         />
143         <span class="mx-4 text-h6">{{ quantity }}</span>
144         <v-btn
145           icon="mdi-plus"
146           size="small"
147           variant="outlined"
148           @click="incrementQuantity"
149         />
150         <v-spacer />
151         <v-btn
152           icon="mdi-delete"
153           color="error"
154           variant="text"
155           @click="emit('remove')"
156         />
157       </div>
158     </v-card-actions>
159   </v-card>
160 </template>
161
```

ProductList.vue

launch.jsonProductList.vue X

frontend > src > components > ProductList.vue > {} script setup

```
1 <script setup lang="ts">
2 import type { Product } from '../types';
3 import ProductCard from './ProductCard.vue';
4
5 interface Props {
6   products: Product[];
7   loading?: boolean;
8 }
9
10 defineProps<Props>();
11 </script>
12
13 <template>
14   <div>
15     <v-row v-if="loading">
16       <v-col v-for="n in 6" :key="n" cols="12" sm="6" md="4">
17         <v-skeleton-loader type="card" />
18       </v-col>
19     </v-row>
20
21     <v-alert v-else-if="products.length === 0" type="info" variant="tonal">
22       No products found matching your filters.
23     </v-alert>
24
25     <v-row v-else>
26       <v-col
27         v-for="product in products"
28         :key="product.id"
29         cols="12"
30         sm="6"
31         md="4"
32       >
33         <ProductCard :product="product" show-add-to-cart />
34       </v-col>
35     </v-row>
36   </div>
37 </template>
38
```

CartView.vue

```
{} launch.json  CartView.vue X
frontend > src > views > CartView.vue > {} script setup
1  <script setup lang="ts">
2  import { computed, ref } from 'vue';
3  import confetti from 'canvas-confetti';
4  import { useCart } from '../composables/useCart';
5  import ProductCard from '../components/ProductCard.vue';
6
7  const { cartItems, updateQuantity, removeFromCart, cartTotal, clearCart } =
8    useCart();
9
10 const showSuccessMessage = ref(false);
11
12 const formattedTotal = computed(() => {
13   return new Intl.NumberFormat('en-US', {
14     style: 'currency',
15     currency: 'USD',
16   }).format(cartTotal.value);
17 });
18
19 const handleCheckout = () => {
20   confetti({
21     particleCount: 100,
22     spread: 70,
23     origin: { y: 0.6 },
24   });
25
26   showSuccessMessage.value = true;
27   clearCart();
28 };
29 </script>
30
31 <template>
32   <v-container>
33     <h1 class="text-h4 mb-6">Shopping Cart</h1>
34
35     <v-alert v-if="cartItems.length === 0" type="info" variant="tonal" class="mb-4">
36       Your cart is empty. Start shopping to add items!
37     </v-alert>
38
39     <template v-else>
40       <v-row>
41         <v-col>
42           <v-for="item in cartItems"
43             :key="item.product.id"
44             cols="12"
45             md="6"
46             lg="3">
47             <ProductCard
48               :product="item.product"
49               :quantity="item.quantity"
50               :show-add-to-cart="false"
51               show-quantity-controls
52               @update:quantity="updateQuantity(item.product.id, $event)"
53               @remove="removeFromCart(item.product.id)"
54             />
49         </v-col>
50       </v-row>
51
52       <v-card class="mt-6 pa-4" elevation="2">
53         <v-card-title class="text-h5">Order Summary</v-card-title>
54         <v-card-text>
55           <div class="d-flex justify-space-between text-h6">
56             <span>Total:</span>
57             <span class="text-primary font-weight-bold">{{ formattedTotal }}</span>
58           </div>
59         </v-card-text>
60       </v-card>
61     </template>
62   </v-container>
63 </template>
```

() launch.json

▼ CartView.vue X

frontend > src > views > ▼ CartView.vue > {} script setup

```
31 <template>
32   <v-container>
39     <template v-else>
40       <v-row>
41         <v-col
42           :key="item.product.id"
43           cols="12"
44           md="6"
45           lg="3"
46         >
47           <ProductCard
48             :product="item.product"
49             :quantity="item.quantity"
50             :show-add-to-cart="false"
51             show-quantity-controls
52             @update:quantity="updateQuantity(item.product.id, $event)"
53             @remove="removeFromCart(item.product.id)"
54           />
55         </v-col>
56       </v-row>
57     </v-container>
58   </v-card>
59   <v-card class="mt-6 pa-4" elevation="2">
60     <v-card-title class="text-h5">Order Summary</v-card-title>
61     <v-card-text>
62       <div class="d-flex justify-space-between text-h6">
63         <span>Total:</span>
64         <span class="text-primary font-weight-bold">{{ formattedTotal }}</span>
65       </div>
66     </v-card-text>
67     <v-card-actions>
68       <v-btn color="error" variant="text" @click="clearCart">
69         Clear Cart
70       </v-btn>
71       <v-spacer />
72       <v-btn color="primary" variant="elevated" size="large" @click="handleCheckout">
73         Checkout
74       </v-btn>
75     </v-card-actions>
76   </v-card>
77 </template>
78
79 <v-snackbar v-model="showSuccessMessage" color="success" :timeout="3000">
80   Order placed successfully! Thank you for your purchase.
81 </v-snackbar>
82 </v-container>
83 </template>
84
```